

DSA Assignment #7: 🌲 (2/3)

2500010774 兰玉琪 数学科学学院

1. 题目

M297.二叉树的序列化与反序列化

dfs, bfs, <https://leetcode.cn/problems/serialize-and-deserialize-binary-tree/>

思路：

代码：

```

class Codec:

    def serialize(self, root):
        """Encodes a tree to a single string.

        :type root: TreeNode
        :rtype: str
        """
        def construct(node):
            if not node:
                return ""
            mystr = f"{node.val}"
            if node.left and node.right:
                mystr += f" ( {construct(node.left)} , {construct(node.right)} )"
            elif node.left:
                mystr += f" ( {construct(node.left)} , )"
            elif node.right:
                mystr += f" ( , {construct(node.right)} )"
            return mystr
        return construct(root)

    def deserialize(self, data):
        """Decodes your encoded data to tree.

        :type data: str
        :rtype: TreeNode
        """
        if not data:
            return None
        mylist = data.split()
        stack = [TreeNode(int(mylist[0]))]
        for i in range(1, len(mylist)):
            obj = mylist[i]
            if obj == "(" or obj == ",":
                stack.append(obj)
            elif obj == ")":
                flagr = True
                rightnode = None
                leftnode = None
                while stack and stack[-1] != "(":
                    t = stack.pop()
                    if t == ',':
                        flagr = False

```

```

        continue
    if flagr:
        rightnode = t
    else:
        leftnode = t
    stack.pop()
    stack[-1].left = leftnode
    stack[-1].right = rightnode
else:
    stack.append(TreeNode(int(obj)))
return stack[0]

```

All Submissions
Python3
Auto

Runtime
99 ms | Beats 19.87%

Memory
22.35 MB | Beats 89.59%

```

1
2
3 class Codec:
4
5     def serialize(self, root):
6         """Encodes a tree to a single string.
7
8         :type root: TreeNode
9         :rtype: str
10
11         """
12     def construct(node):

```

M129.求根节点到叶节点数字之和

dfs, <https://leetcode.cn/problems/sum-root-to-leaf-numbers/>

思路：

代码：

```

class Solution:
    def sumNumbers(self, root: Optional[TreeNode]) -> int:
        if not root:
            return 0
        total = 0
        def dfs(node, path):
            nonlocal total
            if not node.left and not node.right:
                total += int(path)
                return
            if node.left:
                dfs(node.left, path+str(node.left.val))
            if node.right:
                dfs(node.right, path+str(node.right.val))
        dfs(root, str(root.val))
        return total

```

All Submissions

Accepted 108 / 108 testcases passed

cZ88PGJi... submitted at Apr 09, 2026 21:57

Runtime

0 ms | Beats **100.00%**

Memory

19.26 MB | Beats **81.20%**

100%

Python3 Auto

```

1 class Solution:
2     def sumNumbers(self, root: Optional[TreeNode]) -> int:
3         if not root:
4             return 0
5         total = 0
6         def dfs(node, path):
7             nonlocal total
8             if not node.left and not node.right:
9                 total += int(path)
10                return
11            if node.left:

```

Saved

Ln 1, Col

M22158:根据二叉树前中序序列建树

tree, <http://cs101.openjudge.cn/practice/22158/>

思路：

代码：

```

import sys
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
def construct(i, j, k, l, qian, zhong):
    if i > j:
        return
    root = TreeNode(qian[i])
    a = zhong.index(qian[i])
    t = a - k
    root.left = construct(i+1, i+t, k, k+t-1, qian, zhong)
    root.right = construct(i+t+1, j, a+1, l, qian, zhong)
    return root
def houxu(tree):
    if tree:
        houxu(tree.left)
        houxu(tree.right)
        result.append(tree.val)
data = sys.stdin.read().split()
for i in range(len(data)//2):
    qian, zhong = list(data[2*i]), list(data[2*i+1])
    tree = construct(0, len(qian)-1, 0, len(zhong)-1, qian, zhong)
    result = []
    houxu(tree)
    print("".join(result))

```

状态: Accepted

Source Code

```
import sys
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
def construct(i, j, k, l, qian, zhong):
    if i > j:
        return
    root = TreeNode(qian[i])
    a = zhong.index(qian[i])
    t = a - k
    root.left = construct(i+1, i+t, k, k+t-1, qian, zhong)
    root.right = construct(i+t+1, j, a+1, l, qian, zhong)
    return root
def houxu(tree):
    if tree:
        houxu(tree.left)
        houxu(tree.right)
        result.append(tree.val)
data = sys.stdin.read().split()
for i in range(len(data)//2):
    qian, zhong = list(data[2*i]), list(data[2*i+1])
    tree = construct(0, len(qian)-1, 0, len(zhong)-1, qian, zhong)
    result = []
    houxu(tree)
    print("".join(result))
```

基本信息

#: 51702872

题目: 22158

提交人:
25n2500010774(sleepy(25n2500010774))

内存: 3676kB

时间: 26ms

语言: Python3

提交时间: 2026-01-23 20:40:36

M24729:括号嵌套树

dfs, stack, <http://cs101.openjudge.cn/practice/24729/>

思路:

代码:

```

import sys
mystr = sys.stdin.read()
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.children = []
stack = []
for s in mystr:
    if s.isalpha():
        stack.append(TreeNode(s))
    elif s == ")":
        children = []
        while True:
            a = stack.pop()
            if a == "(":
                break
            if a == ",":
                continue
            children.append(a)
        stack[-1].children.extend(children)
    else:
        stack.append(s)
tree = stack[0]
result = []
def qianxu(root):
    if root:
        result.append(root.val)
        for node in reversed(root.children):
            qianxu(node)
result1 = []
def houxu(root):
    if root:
        for node in reversed(root.children):
            houxu(node)
        result1.append(root.val)
qianxu(tree)
houxu(tree)
print("".join(result))
print("".join(result1))

```

状态: **Accepted**

Source Code

```
import sys
mystr = sys.stdin.read()
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.children = []
stack = []
for s in mystr:
    if s.isalpha():
        stack.append(TreeNode(s))
    elif s == "(":
        children = []
        while True:
            a = stack.pop()
            if a == "(":
                break
            if a == ",":
                continue
            children.append(a)
        stack[-1].children.extend(children)
    else:
        stack.append(s)
tree = stack[0]
result = []
def qianxu(root):
```

基本信息

#: 51697337

题目: 24729

提交人:
25n2500010774(sleepy(25n2500010774))

内存: 3664kB

时间: 25ms

语言: Python3

提交时间: 2026-01-22 13:51:45

M01577: Falling Leaves

tree, <http://cs101.openjudge.cn/25dsapre/solution/51728513/>

思路:

代码


```

import sys
data = sys.stdin.read().split()
i = 0
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
seq = []
while i < len(data):
    if data[i] == "$" or data[i] == "*":
        def insert(node, s):
            if not node:
                node = TreeNode(s)
                return
            cur = node
            while cur:
                if s < cur.val:
                    cur1 = cur
                    cur = cur.left
                    flag = True
                else:
                    cur1 = cur
                    cur = cur.right
                    flag = False
            if flag:
                cur1.left = TreeNode(s)
            else:
                cur1.right = TreeNode(s)
        tree = TreeNode(seq.pop())
        while seq:
            for s in list(seq.pop()):
                insert(tree, s)
        result = []
        def preorder_traversal(node):
            if node:
                result.append(node.val)
                preorder_traversal(node.left)
                preorder_traversal(node.right)
        preorder_traversal(tree)
        print("".join(result))
        seq = []
        if data[i] == "$":

```

```
        break
    else:
        seq.append(data[i])
    i += 1
```

#51728513提交状态

[View](#)

[Submit](#)

[Statistics](#)

[Clarify](#)

状态: **Accepted**

Source Code

```
import sys
data = sys.stdin.read().split()
i = 0
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
seq = []
while i < len(data):
    if data[i] == "$" or data[i] == "*":
        def insert(node, s):
            if not node:
                node = TreeNode(s)
                return
            cur = node
            while cur:
                if s < cur.val:
                    cur1 = cur
                    cur = cur.left
                    flag = True
                else:
```

基本信息

#: 51728513

题目: 01577

提交人: 25n2500010774(sleepy(25n2500010774))

内存: 3652kB

时间: 25ms

语言: Python3

提交时间: 2026-01-29 13:38:15

1843D. Apple Tree

Combinatorics, dfs and similar, dp, math, trees, 1200,

<https://codeforces.com/problemset/problem/1843/D>

思路:

代码

```
import sys
from collections import defaultdict
sys.setrecursionlimit(300000)
for _ in range(int(input())):
    dingdianshu = int(input())
    shu = defaultdict(list)
    while True:
        mylist = list(map(int, input().split()))
        if len(mylist) == 1:
            q = mylist[0]
            break
        a, b = mylist
        shu[a].append(b)
        shu[b].append(a)
    visited = [0] * (dingdianshu + 1)
    dp = [0] * (1 + dingdianshu)
    def dfs(i):
        visited[i] = 1
        sum = 0
        for num in shu[i]:
            if visited[num] == 0:
                sum += dfs(num)
        if sum == 0:
            sum = 1
        dp[i] = sum
        return sum
    dfs(1)
    for t in range(q):
        a, b = map(int, input().split())
        print(dp[a] * dp[b])
```

General

#	Author	Problem	Lang	Verdict	Time	Memory	Sent	Judged		
355008386	Practice: lyqpk	1843D - 60	Python 3	Accepted	1500 ms	81764 KB	2025-12-25 08:20:07	2025-12-25 08:20:07		<button>Compare</button>

→ SourceCopy

```
import sys
from collections import defaultdict
sys.setrecursionlimit(300000)
for _ in range(int(input())):
    dingdianshu = int(input())
    shu = defaultdict(list)
    while True:
        mylist = list(map(int, input().split()))
        if len(mylist) == 1:
            q = mylist[0]
            break
        a, b = mylist
        shu[a].append(b)
        shu[b].append(a)
    visited = [0] * (dingdianshu + 1)
    dp = [0] * (1 + dingdianshu)
    def dfs(i):
        visited[i] = 1
        sum = 0
        for num in shu[i]:
            if visited[num] == 0:
                sum += dfs(num)
        if sum == 0:
            sum = 1
        dp[i] = sum
    return dp[1]
dfs(1)
```

2. 学习总结和个人收获

每日选做然后就没了，期中了还是很忙的